

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

ОТЧЁТ

Домашняя работа 5

Межсервисное взаимодействие через RabbitMQ

Выполнил:
Прель Александр
Группа БР1.1

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Подключить и настроить RabbitMQ или Kafka. Реализовать межсервисное взаимодействие через выбранный брокер и продемонстрировать результат на приложении бронирования столиков.

Ход работы

1. Исходная архитектура и изменение

Д35 развивает микросервисы ЛР2 в labs/lab2: Accounts, Catalog, Bookings и Gateway. Добавлен отдельный Notifications со своей SQLite-базой. Исходный монолит labs/lab1 не изменён. Выбран RabbitMQ 4.3.5; брокер работает локально в Docker Desktop, а сервисы - процессами Node.js на Mac.

Участник	Действие и данные
Bookings	Сохраняет бронь и событие в reservations + outbox_events одной операцией SQLite.
Publisher	Читает outbox, публикует событие через AMQP и ждёт publisher confirm.
RabbitMQ	Durable topic exchange направляет сообщения в durable quorum queue.
Notifications	Проверяет событие, записывает уведомление в свою SQLite и подтверждает доставку (ack).
Gateway / клиент	GET /notifications/me с JWT возвращает только уведомления текущего пользователя.

2. Контракт события

События: reservation.created и reservation.cancelled. Общие поля: eventId, schemaVersion=1, type, occurredAt, data. Данные включают ID брони, пользователя, ресторана и столика; снимки названия и номера; дату, интервал, число гостей и статус. JWT, пароли и email не передаются.

eventId - 32 шестнадцатеричных символа; AMQP messageId совпадает с ним, routing key совпадает с type. Размер сообщения ограничен 64 КБ. Проверяются версия, типы, существующая дата, время и соответствие статуса событию. Неизвестные дополнительные поля допустимы.

3. Асинхронность

Клиент получает результат создания брони, не дожидаясь уведомления. При остановленном RabbitMQ события остаются в outbox. После восстановления соединения publisher отправляет их. Уведомления - локальные записи API, реальные email и SMS не отправляются.

4. Надёжность и обработка ошибок

SQLite-триггер обеспечивает атомарность брони и outbox. Если запись события не удалась, изменение брони откатывается. Publisher помечает событие опубликованным только после подтверждения брокера. Сообщения persistent, публикация mandatory.

Consumer использует prefetch=1 и manual ack после записи. event_id - первичный ключ уведомления: ON CONFLICT DO NOTHING не создаёт дубликат при повторной доставке, в том числе после перезапуска.

Ситуация	Результат
Потеря связи	Переподключение с паузой 500 мс; неподтверждённый outbox остаётся в SQLite.
Ошибка записи получателя	До 3 повторов с паузой 300 мс; максимум 4 попытки обработки.
Неверное событие	Сразу в очередь ошибок, x-error=INVALID_EVENT.
Повторы исчерпаны	В очередь ошибок, x-error=PROCESSING_FAILED; сообщение сохранено для диагностики.
Повторная отмена / конфликт	Нет успешного изменения брони, поэтому лишнего события нет.

Повтор и публикация в очередь ошибок подтверждаются перед ack исходной доставки. Очередь ошибок объявляется явно; перенос выполняет приложение. Гарантия - at-least-once с идемпотентной записью, а не exactly-once. При повторях порядок может измениться; уведомления хранят отдельные события, не восстанавливают по порядку текущее состояние брони.

5. Проверка существующего функционала

Прогон	Фактический итог 10.09.2026
npm ci --include=dev	308 пакетов; exit 0. Препрежние версии не обновлялись.
Typecheck и build	Успешно; запуск из dist после сборки tsc.
npm test	22/22: 17 HTTP-сценариев и 5 тестов валидатора событий.
npm run test:messaging	10/10: реальные AMQP, HTTP, SQLite и перезапуски.
Newman на Д35 и ЛР1	На каждой версии 29 запросов, 117 assertions, 0 ошибок.
HTTP smoke Д35	Health, ready, каталог, регистрация, вход, профиль, бронь, отмена и два уведомления.

Проверены все существующие группы API: пользователи, каталог, отзывы, бронирования, права и валидация. Дополнительно: 12 одновременных попыток занять столик, отказы сервисов, HTTP-таймаут и сохранение баз после перезапуска. Это набор конкретных сценариев, не доказательство отсутствия всех ошибок.

6. Проверки отказов RabbitMQ

Десять интеграционных сценариев проверяют: создание/отмену; дубликаты; накопление при остановленном consumer; перезапуск брокера; повторную доставку без ask; остановку брокера и publisher; неверный JSON и версию; сбой SQLite с повторами и восстановлением; атомарный откат; приватность уведомлений. Полный перечень находится в TEST_RESULTS.md.

Первый прогон выявил смену host-порта Docker после restart. Порт теперь выбирается заранее и закрепляется. Затем исправлено завершение AMQP-канала тестового клиента перед закрытием соединения, чтобы ask не терялся между тестами. Финальный полный прогон прошёл 10/10.

7. Воспроизводимый запуск

Из каталога БР1.1/Прель Александр/labs/lab2:

```
npm ci --include=dev
npm run setup:messaging
npm run broker:start
npm run typecheck
npm run build
npm run start:messaging
```

Фактический Gateway: <http://127.0.0.1:8084>. Сервисы: 8085-8088. AMQP: 53277; RabbitMQ Management: 53278. Все привязки только к 127.0.0.1. Новая .env.hw5 хранит случайные секреты и отдельные SQLite-пути; старые .env и базы не перезаписаны. Образ RabbitMQ закреплён digest; данные брокера сохраняются в отдельном Docker volume.

Проверки: npm test; npm run test:messaging; npm run test:postman:messaging; npm run test:smoke:messaging. Остановка API: npm run stop:messaging или Ctrl-C в терминале ручного запуска; брокера: npm run broker:stop. Повторный запуск: broker:start и start:messaging. Интеграционные тесты удаляют только свои временные ресурсы.

Вывод

Реализован и проверен обмен событиями между независимыми Bookings и Notifications через настоящий локальный RabbitMQ. Добавлены outbox, подтверждения, защита от дубликатов, ограниченные повторы и очередь ошибок. Прежний публичный контракт сохранил работоспособность. Код и отчёты находятся в локальной ветке codex/local-setup, базовый commit 922eba1; commit и push не выполнялись.

Ограничения: один узел, нет TLS, production-нагрузки, автоматической очистки outbox и защиты от потери диска. npm audit сообщает 14 проблем прежних зависимостей, включая одну critical; автоматические обновления не выполнялись. Следующий этап - ЛР3: контейнеризация сервисов и Docker Compose. Защита перед преподавателем отдельно не подтверждается.

Материалы и источники

homeworks/hw5: README.md, openapi.yaml, TEST_RESULTS.md.

Документация: rabbitmq.com/docs/confirmations; rabbitmq.com/docs/dlx; amqp-node.github.io/amqplib/channel_api.html.

Среда: macOS arm64, Node.js 24.14.0, npm 11.9.0.